Secure Coding Review Report

Project: Secure Review of Python Login System
Prepared by: E Arvind Udiyar
Course: MSc Computer Science

---

1. Introduction

Secure coding refers to writing software that protects against vulnerabilities and cyber threats. This report presents a security review of a Python-based login system, comparing an insecure version with an improved secure version.

2. Application Overview

The application is a console-based login system implemented using Python. The insecure version stores credentials in plain text and allows direct comparison of passwords. The secure version enhances protection by implementing password hashing using SHA-256 and limiting login attempts.

3. Identified Vulnerabilities

Hardcoded Credentials (High Risk): The username and password are written directly in the source code, which makes them visible if the file is accessed.

Plain Text Password Storage (High Risk): Storing passwords in readable format increases the risk of credential theft.

No Brute Force Protection (Medium Risk): Unlimited login attempts allow attackers to repeatedly try passwords.

4. Security Improvements

Password Hashing: The secure version converts passwords into SHA-256 hash values before comparison. This prevents storing actual passwords in plain text.

Limited Login Attempts: The system restricts login attempts to three tries. If exceeded, the account is locked to prevent brute-force attacks.

5. Execution Screenshots

insecure_login.py - C:/Users/ADMIN/Documents/Arvind/insecure_login.py (3.13.7)

File Edit Format Run Options Window Help

```python
# Insecure Login System

stored_username = "admin"
stored_password = "12345"

username = input("Enter username: ")
password = input("Enter password: ")

if username == stored_username and password == stored_password:
    print("Login Successful!")
else:
    print("Invalid Credentials")
```

Ln: 13 Col: 0

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ADMIN/Documents/Arvind/insecure_login.py ==========
Enter username: admin
Enter password: 12345
Login Successful!
>>>
```

Ln: 8 Col: 0

secure_login.py - C:/Users/ADMIN/Documents/Arvind/secure_login.py (3.13.7)

File Edit Format Run Options Window Help

```python
import hashlib

# Store hashed password instead of plain text
stored_username = "admin"

# Hash of "StrongPass123"
stored_password_hash = hashlib.sha256("StrongPass123".encode()).hexdigest()

attempts = 3

while attempts > 0:
    username = input("Enter username: ")
    password = input("Enter password: ")

    password_hash = hashlib.sha256(password.encode()).hexdigest()

    if username == stored_username and password_hash == stored_password_hash:
        print("Login Successful!")
        break
    else:
        attempts -= 1
        print("Invalid Credentials")
        print("Attempts left:", attempts)

if attempts == 0:
    print("Account Locked. Too many failed attempts.")
```

Ln: 27 Col: 0

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/ADMIN/Documents/Arvind/insecure_login.py ==========
Enter username: admin
Enter password: 12345
Login Successful!
>>>
=========== RESTART: C:/Users/ADMIN/Documents/Arvind/secure_login.py ===========
Enter username: admin
Enter password: 12345
Invalid Credentials
Attempts left: 2
Enter username: admin
Enter password: StrongPass123
Login Successful!
>>>
```

Ln: 17 Col: 0

6. Conclusion

This secure coding review demonstrates how implementing password hashing and limiting login attempts significantly improves application security. Adopting secure coding practices helps protect systems from unauthorized access and strengthens overall software reliability.